

Travaux dirigés n° 3

Processus et tubes

Exercice 1 (Calcul distribué)

Soit un programme séquentiel permettant de calculer la valeur $f(x) + g(x)$ pour un x choisi. f et g sont deux fonctions qui prennent en paramètre un entier et qui retournent un entier. Nous souhaitons réaliser les calculs de chacune des fonctions dans 2 processus distincts pour accélérer les calculs.

Nous proposons la solution suivante (sans les inclusions) :

```
int f(int x) { /* Code de f */ }
int g(int x) { /* Code de g */ }

int main() {
    int n, v = 0, x = 3;

    if((n = fork()) == -1) {
        perror("Erreur lors de la creation du fils");
        exit(EXIT_FAILURE);
    }
    if(n == 0)
        v = v + f(x);
    else
        v = v + g(x);
    printf("Valeur de v: %d\n", v);

    return EXIT_SUCCESS;
}
```

1. Expliquez pourquoi cette solution n'est pas satisfaisante.
2. Corrigez-la pour obtenir le résultat attendu.
3. Est-ce que cette solution (corrigée et fonctionnelle!) apporte forcément des améliorations par rapport à une version séquentielle ?

Exercice 2 (Le jeu de la vie)

Nous considérons le jeu de la vie représenté par une grille de 20 colonnes par 10 lignes où chaque case peut contenir 0 (la cellule est morte) ou 1 (la cellule est en vie). Elle est initialisée avec n cellules en vie. Pour simuler l'étape suivante, nous suivons les règles suivantes :

- Si une cellule morte possède exactement 3 cellules voisines vivantes (parmi les 8 cases adjacentes), elle devient vivante ;
- Si une cellule est vivante, elle le reste si elle possède 2 ou 3 cellules voisines vivantes.

Nous supposons l'existence de la procédure `jeu_vie` qui simule le jeu de la vie. Elle prend en paramètre une grille de 20 par 10 et la modifie suivant les règles précédentes.

Nous souhaitons écrire un programme `ncurses` qui permet de simuler en parallèle 4 jeux de la vie dans 4 fenêtres différentes. Pour cela, il crée 4 fils différents et chacun lui envoie indéfiniment l'état de la simulation (une grille de 20 par 10) via un tube par processus, toutes les secondes.

1. Représentez l'architecture de l'application.

2. Combien de descripteurs de fichier sont nécessaires ?
3. Donnez le code permettant de créer les tubes nécessaires.
4. Donnez le code permettant de créer les 4 fils. Vous n'oublierez pas de fermer tous les descripteurs de fichier inutiles.
5. Expliquez ce que fait la boucle principale du père.

Nous considérons maintenant que la simulation du jeu de la vie est un programme indépendant nommé `vie`.

6. Comment le fils du programme peut récupérer ce qui est généré par le programme `vie` ?
7. Donnez le code du fils.

Exercice 3 (*Enter the Matrix*)

Nous souhaitons réaliser une application qui crée n fils communiquant entre eux via des tubes anonymes. Nous proposons de créer une paire de tubes pour chaque couple de fils.

- 1°) Avec n fils, combien de tubes sont nécessaires au total ? Combien de descripteurs de fichiers sont nécessaires pour les tubes pour chaque fils ?
- 2°) Pourquoi est-il préférable d'utiliser une paire de tubes entre chaque paire de fils au lieu d'un seul ?
- 3°) Que se passe-t-il si le nombre de fils devient trop important ?
- 4°) Comment un fils peut-il écouter dans l'ensemble de ses tubes ? Proposez une solution en précisant les appels systèmes nécessaires.

L'ensemble des tubes est représenté par une matrice de $n \times n$ où chaque case est un tableau de 2 cases. Si on considère que les processus sont numérotés de 0 à $n - 1$, pour le fils i , la ligne i contient les tubes en écriture vers les autres fils et la colonne i contient les tubes en lecture depuis les autres fils. La diagonale n'est pas utilisée.

- 5°) Donnez la portion de code permettant de créer les tubes et les fils (n est spécifié comme une constante). Les descripteurs inutiles doivent être fermés.
- 6°) Nous souhaitons maintenant laisser la possibilité au père d'écrire des messages à ses fils en utilisant les tubes déjà créés. Expliquez les problématiques et proposez une solution pour les limiter autant que possible, sans créer de nouvelles ressources systèmes.